

ERP-Core: 从 12 导联扩展到 14/21 导联的 A/N/D 实验

0. 动机：是“没补好”，还是“补了也没用”？

当 Dynamic 补全没有带来分类提升时，首先需要拆开两个问题：

- 1. **重建问题**：缺失导联的 latent 是否没有补好？
- 2. **分类问题**：即使 latent 补得更准，补出的信息是否真的能帮助分类？

本节先分析第一个问题：误差是否集中在少数难补样本，以及这些样本有什么特征。后面的 A/N/D 对照再回答第二个问题。

范围说明：下面的长尾、分导联 MSE 和 roughness 统计都是针对原始 **12 观测** → **28 目标 (16 缺失)** 设置做的诊断。它们是设计 14/21 导联实验的动机，不能直接当作 14/21 导联的诊断结果。

0.1 缺失 latent 误差存在中等程度长尾

【误差是否集中在少数难补样本，以及这些样本有什么特征？】

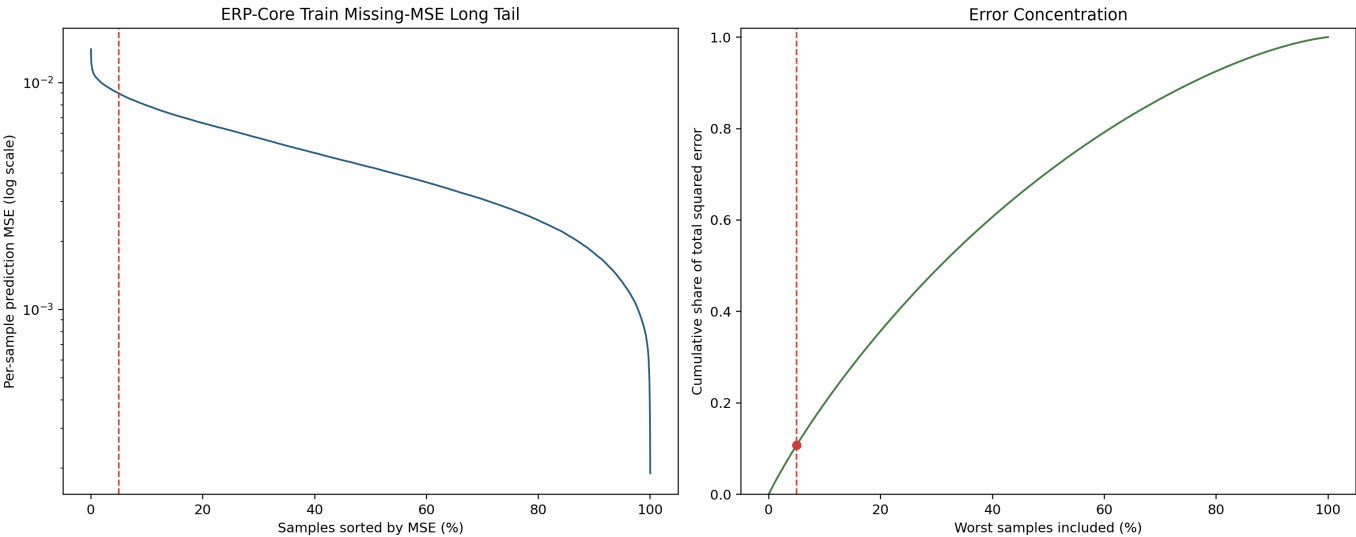


Figure 1: ERP-Core 样本级缺失 latent MSE 长尾与误差集中度

对每个训练样本计算：

$$\text{sample MSE} = \text{MSE}(\text{p_miss} + \text{d_sub} + \text{d_task}, \text{h_miss_target})$$

左图将样本按 MSE 从大到小排序：横轴 0% 一端是最难样本，100% 一端是最易样本；纵轴是对数坐标下的 MSE。结果为：

- Worst 5% 共 2,498 个样本，分界阈值为 $\text{MSE} \geq 0.008975$ 。
- 样本最大 MSE 为 0.014051，中位数为 0.004242。
- 曲线没有突然断崖：误差不是被极少数异常点完全主导，但确实存在一段困难样本长尾。

右图统计最难样本对全部 squared error 的累积贡献：

纳入的困难样本	占总 squared error 的比例
Worst 1%	2.43%
Worst 5%	10.81%
Worst 10%	19.98%

如果误差在所有样本中完全均匀，Worst 5% 应只贡献 5%；实际贡献了 10.81%，是均匀情况的 2.16×。绝对误差也有同样趋势：

样本分组	Mean MSE
全部 Train	0.004589
Worst 5%	0.009921

样本分组	Mean MSE
其余 95%	0.004309

Worst 5% 的平均 MSE 是其余 95% 的约 2.30×。因此，当前更准确的描述是：**大多数样本的补全误差相对稳定，但存在一批持续偏难的样本；它们放大了总误差，但不是唯一来源。**

0.2 困难误差更集中在哪些导联？

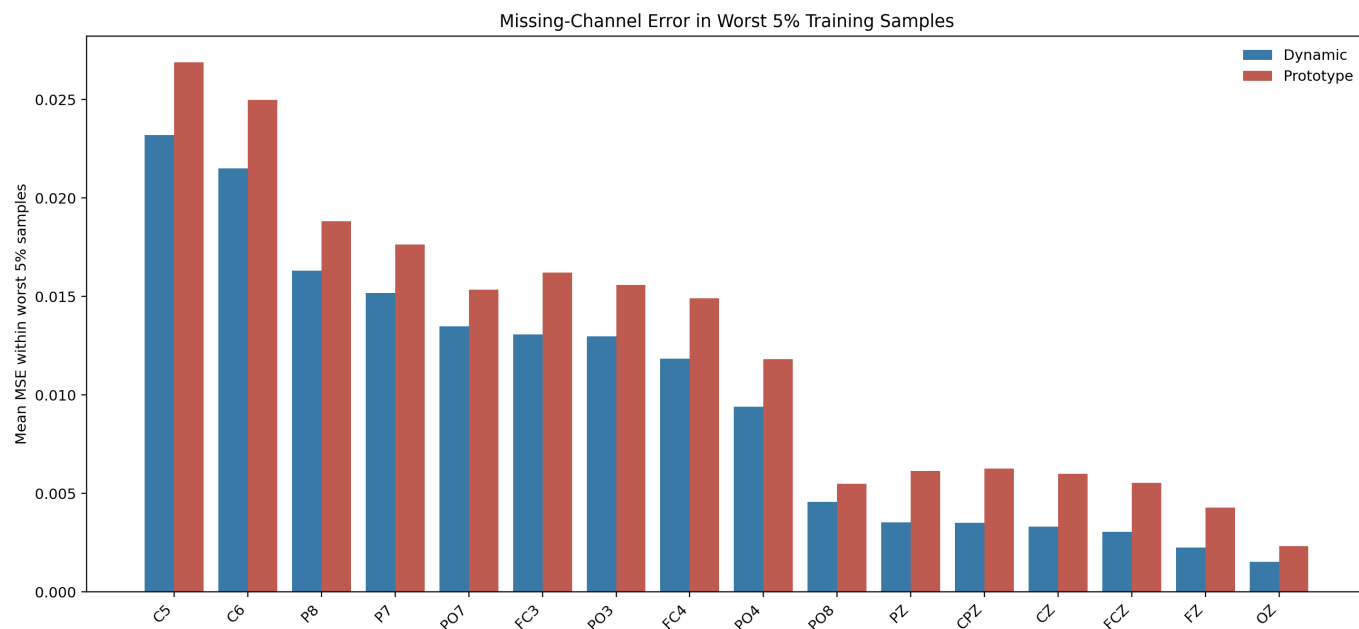


Figure 2: Worst 5% 样本中各缺失导联的 Dynamic/Prototype MSE

在 Worst 5% 样本中，误差最高的前五个缺失导联为：

导联	Dynamic MSE	Prototype MSE	Dynamic 是否更好
C5	0.02319	0.02689	是
C6	0.02150	0.02499	是
P8	0.01632	0.01883	是
P7	0.01517	0.01764	是
PO7	0.01348	0.01535	是

这里有两个不同层面的结论：

1. C5/C6/P8/P7/PO7 等导联在困难样本中更难预测，但误差不是由单一导联造成。
2. 这些导联上 Dynamic MSE 仍低于 Prototype MSE，因此不是 Dynamic correction 把它们“补坏了”；更合理的说法是，Dynamic 有改善，但没有完全克服这些样本本身的高难度。

0.3 为什么这些样本难补？与时间粗糙度的关系

时间粗糙度是什么？ 这是一个**事后诊断指标**，不是训练 loss，也没有作为模型输入。它在 z-score 后的真实缺失导联 EEG 上计算。对于某个缺失导联 c：

$$\text{roughness}(c) = \text{RMS}(x[c, t] - x[c, t-1]) / (\text{RMS}(x[c, t]) + \text{eps})$$

【解释一下这个公式】相对于信号本身的幅度，相邻时间点变化得有多剧烈。数值越大，信号通常越不平滑。

- 分子表示相邻采样点的变化幅度。
- 分母表示该段信号自身的整体幅度。

- 两者相除是为了衡量“相对于自身幅度，波形在时间上变化得有多快”。

因此，roughness 高不是说 EEG 整体幅值一定更大，而是说波形更锯齿、相邻时刻变化更剧烈。

观察到了什么？

指标	结果	直接含义
Val Worst 5% 的平均 roughness	0.936	困难样本的时间变化更剧烈
Val 其余 95% 的平均 roughness	0.527	作为普通样本参照
Worst 5% 中同时属于 roughness 最高 5% 的比例	53.96%	超过一半的高误差样本也是高粗糙样本
两组无关时的随机重合预期	5%	作为富集的零假设参照
相对随机预期的富集	10.79×	53.96% / 5%
roughness 与重建 MSE 的 Spearman 相关	Val 0.848; Train 0.881	roughness 越高，MSE 通常也越高

Val Worst 5% 的平均 roughness 比其余样本高约 77.5%。再结合 0.848 的 Spearman 相关和 10.79× 的富集，可以得到一个比较稳健的**相关性结论**：时间变化越剧烈的缺失导联信号，其 latent 越容易补不准。

【这一块我该怎么说？】

为什么 roughness 高可能使补全更难？ 下面是对现有结果的机制解释，还不是已被实验证明的因果：

1. **快速变化更难从其他导联推断。**缓慢、大尺度的 ERP 成分往往在多个导联上共享；某个缺失导联上短暂、局部的快速变化，可能不能由其他观测导联唯一决定。
2. **Prototype 代表典型平均模式。**p_miss 是静态原型，更容易表示稳定、共性的成分，不容易包含当前单个 trial 中的瞬时快变。
3. **Correction 的表达幅度受限。**当前 $d_{\text{sub}}/d_{\text{task}} = 0.02 * \tanh(\text{raw})$ ，每个分支的逐元素幅度被限制在 $(-0.02, 0.02)$ 。如果目标 token 的快变部分需要更大或更细的样本级修正，这个边界可能限制补全。
4. **CNN token 仍会反映时间形态。**虽然训练的 missing MSE 在 CNN latent 上计算，不是原始波形 MSE，但 patch embedding 由时间波形产生。原始波形中难以预测的快变部分仍可能表现为更难匹配的 target token。

这些结果现在能说明什么，不能说明什么？ 可以说：

- 高 roughness 与高 missing-latent MSE 存在强相关。
- 困难样本显著富集于 roughness 最高的样本群。
- Dynamic 相对静态 Prototype 仍改善了这些导联的 MSE，但剩余误差仍与快变信号相关。

暂时不能说：

- roughness 是高重建误差的唯一原因或直接原因。
- roughness 高的样本一定是噪声或坏数据。快变可能来自真实高频成分，也可能来自肌电/眼电伪迹或预处理效应。
- 只要增加这些真实导联，下游分类就一定改善。补全 MSE 和分类 BAcc 是两个不同层面的指标。

我们想知道，哪些样本的缺失导联更难补全，因此用时间粗糙度衡量真实 EEG 在相邻时间点上的变化程度。

在验证集中，重建误差最高的 5% 样本，平均粗糙度为 0.936，其余样本为 0.527，前者高约 77.5%。这说明，高误差样本的真实缺失导联往往波动更剧烈。

进一步看，这些高误差样本中，有 53.96% 同时属于粗糙度最高的 5%；如果两者无关，预期只有约 5%，实际重合比例是这个预期的 10.79 倍。

而且，这个关系不只存在于少数极端样本：在整个验证集上，粗糙度与重建 MSE 的 Spearman 相关达到 0.848，说明粗糙度较高的样本，重建误差通常也排得更高。

因此，当前结果支持：真实缺失导联的时间变化越剧烈，模型的 latent 补全误差通常越大。

0.4 从诊断到 14/21 导联实验

因此，将误差较高的缺失导联改为真实可观测导联，再检查重建与分类是否改善。

设置	在原 12 导联上新增	真实观测数	仍需补全数
12 导联基线	—	12	16
14 导联	C5, C6	14	14

设置	在原 12 导联上新增	真实观测数	仍需补全数
21 导联	C5, C6, P8, P7, P07, FC3, P03, FC4, P04	21	7

将难补导联改为真实观测

-> missing latent 是否更准？

-> A/N/D 的 Val/Test BAcc 是否改善？

1. 实验目的

在原 ERP-Core 12 导联设置上构建两个逐步增加真实观测的设置：

- 14 导联：新增 C5/C6，仍需补全 14 导联。
- 21 导联：在 14 导联上继续增加 P8/P7/P07/FC3/P03/FC4/P04，仍需补全 7 导联。

每个导联数设置再比较：

- N：只使用真实观测导联，不补全。
- A：真实观测导联 + 静态 prototype，补全到 28 导联。
- D：真实观测导联 + Dynamic Corrector 修正后的 prototype，补全到 28 导联。

2. 14/21 导联定义

2.1 14 导联

可观测导联：

FP1, FP2, F3, F4, F7, F8,
C3, C4, C5, C6,
P3, P4, O1, O2

相对完整 28 导联，14 个缺失导联是：

FC3, P7, P07, P03, OZ, PZ, CPZ,
FZ, FC4, FCZ, CZ, P8, P08, P04

代码定义在 Channels_definition.py 的 ERPCORE_14_CHANNELS。启用参数为：

```
CHANNEL_SUBSET=erpcore14
COMPLETION_SCOPE=erpcore14_with_erpcore28
```

2.2 21 导联

可观测导联：

FP1, FP2, F3, F4, F7, F8,
C3, C4, C5, C6, P3, P4, O1, O2,
P8, P7, P07, FC3, P03, FC4, P04

相对完整 28 导联，剩余 7 个缺失导联是：

OZ, PZ, CPZ, FZ, FCZ, CZ, P08

代码定义在 Channels_definition.py 的 ERPCORE_21_CHANNELS。启用参数为：

```
CHANNEL_SUBSET=erpcore21
COMPLETION_SCOPE=erpcore21_with_erpcore28
```

2.3 Loader 如何选择 12/14/21/28 导联

Loader 中：

```
self.use_full_input = self.channel_names == self.full_channel_names
self.observed_channel_names = list(
    ERPCORE_12_CHANNELS if self.use_full_input else self.channel_names
)
```

当 channel_subset=erpcore14 或 erpcore21 时, self.channel_names 不等于完整 28 导联, 因此 use_full_input=False, 最终 observed_channel_names=self.channel_names, 即当前选中的 14 或 21 导联。

Loader 最终通过:

```
x = x_full if self.use_full_input else x_obs
```

实现:

channel_subset	x_obs	x_full	模型主输入 x
erpcore12	12 导联	28 导联	12 导联
erpcore14	14 导联	28 导联	14 导联
erpcore21	21 导联	28 导联	21 导联
erpcore28	保留的 12 导联视图	28 导联	28 导联

3. Dynamic Stage1 中间流程

3.1 输入与 CNN token

14 导联时 Loader 输出:

```
x_obs [B,14,200]
x_full [B,28,200]
```

进入 engine 后增加 patch 维度:

```
x_obs [B,14,1,200]
x_full [B,28,1,200]
```

冻结的 patch_embed 分别提取 token:

```
h_obs [B,14,1,200]
h_full [B,28,1,200]
```

这里最后一维 200 已经是 CNN token embedding, 不再是原始波形的 200 个采样点。

3.2 找出观测和缺失位置

模型将 14 个真实导联映射到完整 28 导联空间, 得到:

```
obs_indices 14 个位置
miss_indices 14 个位置
```

再从 28 导联 prototype 中抽取缺失部分:

```
p_all [B,28,1,200]
p_miss [B,14,1,200]
```

12 导联版对应的 p_miss 是 [B,16,1,200]。

3.3 Corrector 预测两个 correction

Corrector 输入是观测 token 和缺失 prototype 的拼接:

```
h_obs [B,14,1,200] + p_miss [B,14,1,200]
-> flatten/concat
tokens [B,28,200]
```

随后经过:

```
shared_encoder
|-- subject_encoder -> subject_missing -> d_sub
`-- task_encoder    -> task_missing    -> d_task
```

当前 correction 有 tanh 和 correction_scale=0.02:

```
d_sub = 0.02 * tanh(subject_missing)
d_task = 0.02 * tanh(task_missing)
```

形状都是：

```
d_sub [B,14,1,200]
d_task [B,14,1,200]
```

最终预测：

```
h_pred_miss = p_miss + d_sub + d_task
```

3.4 构造真实缺失导联目标

从 `h_full` 中只取 `miss_indices` 对应的导联：

```
with torch.no_grad():
    h_full = patch_embed(x_full)
    h_miss_target = h_full[:, miss_indices]
```

14 导联版的目标形状：

```
h_miss_target [B,14,1,200]
```

`torch.no_grad()` 表示目标分支不参与反向传播。

4. Stage1 loss 如何计算

当前 `scripts/erp_core/D/stage1.sh` 默认总 loss 为：

```
total_loss = (
    20.0 * loss_missing
    + 0.001 * loss_reg
    + 0.0 * loss_subject_summary_contra
    + 0.0 * loss_task_summary_contra
    + 0.005 * loss_subject_correction_contra
    + 0.005 * loss_task_correction_contra
    + 5.0 * loss_permute_sub
    + 5.0 * loss_permute_task
)
```

4.1 缺失导联重建

```
loss_missing = MSE(h_pred_miss, h_miss_target)
```

即比较：

```
p_miss + d_sub + d_task
```

vs.

真实缺失导联的 CNN token

这是 CNN token MSE，不是原始 EEG 波形 MSE。

4.2 Correction 正则

```
loss_reg = mean(d_sub ** 2) + mean(d_task ** 2)
```

目的是限制 correction 的整体幅度。

4.3 Subject/task correction 对比学习

额外抽取同 subject 或同 task 的样本对：

```
loss_subject_correction_contra = InfoNCE(flatten(d_sub_left), flatten(d_sub_right))
loss_task_correction_contra = InfoNCE(flatten(d_task_left), flatten(d_task_right))
```

目标是使同 subject 的 `d_sub` 更接近，同 task 的 `d_task` 更接近。`z_sub/z_task summary` 对比项的当前权重为 0，不影响优化。

4.4 Subject 交换重建

对同一 subject 的两个样本，交换 d_sub:

```
pred_left  = p_miss_left  + d_sub_right + d_task_left
pred_right = p_miss_right + d_sub_left  + d_task_right

loss_permute_sub = 0.5 * (
    MSE(pred_left, target_left) + MSE(pred_right, target_right)
)
```

4.5 Task 交换重建

对同一 task 的两个样本，交换 d_task:

```
pred_left  = p_miss_left  + d_sub_left  + d_task_right
pred_right = p_miss_right + d_sub_right + d_task_left

loss_permute_task = 0.5 * (
    MSE(pred_left, target_left) + MSE(pred_right, target_right)
)
```

4.6 Stage1 checkpoint 选择

Val 使用与 Train 相同的加权总 loss:

```
current_score = val_stats["loss"]
if current_score < best_score:
    save("checkpoint-best.pth")
```

因此 Stage1 选择的是**最低 Val 加权总 loss**，不是单独最低 loss_missing。Test loss 不参与 checkpoint 选择。

需要注意：Val 的 subject/task pair 也是随机采样，所以 correction contrastive 和 permutation 项会带来一定的 Val loss 波动。

5. outputs/erpcore 当前 A/N/D 结果

以各运行的**最佳 Val BAcc epoch 对应的 Test 结果**为准，不用 Test 选 checkpoint。以下是 2026-09-15 当前文件的快照。

5.1 14/21 导联 A/N/D 主表与 README 中的 12 导联参考

方法	输入与补全	Seed 状态	Best epoch	Val BAcc	Test Acc	Test BAcc	Test Kappa	Test Weighted F1
N14-freeze cmn	14 个真实导联，不补全	0 已完成	1	38.29%	59.18%	39.06%	0.4850	54.95%
A14	14 真实 + 14 静态 prototype	0 已完成	6	37.56%	56.35%	40.14%	0.4662	54.85%
D14	14 真实 + 14 动态修正 prototype	1 已完成	1	37.15%	58.06%	36.57%	0.4709	54.05%
N21-freeze CNN	21 个真实导联，不补全	0 已完成	4	41.07%	58.18%	42.08%	0.4886	55.71%
A21-freeze CNN	21 真实 + 7 静态 prototype	0 已完成	4	41.84%	58.07%	41.00%	0.4884	55.80%
D21	21 真实 + 7 动态修正 prototype	— 未运行	—	—	—	—	—	—
N12-freeze CNN (README)	12 个真实导联，不补全	0 历史已完成	1	38.67%	60.59%	39.73%	0.5008	55.62%
A12-freeze CNN (README)	12 真实 + 16 静态 prototype	0 历史已完成	1	38.46%	61.38%	40.29%	0.5065	56.27%

方法	输入与补全	Seed 状态	Best epoch	Val BAcc	Test Acc	Test BAcc	Test Kappa	Test Weighted F1
O28-freeze CNN (README)	28 个真实导联	0 历史已完成	6	44.74%	57.60%	44.36%	0.4907	57.54%
N12-full finetune (README)	12 个真实导联, 不补全	0 历史已完成	4	45.53%	64.42%	45.42%	0.5488	59.78%
O28-full finetune (README)	28 个真实导联	0 历史已完成	1	53.99%	64.61%	48.91%	0.5612	62.53%
D12 (README/current dynamic)	12 真实 + 16 动态修正 prototype	1 历史已完成	23	36.60%	54.66%	39.61%	0.4445	53.96%
CSLP (README 论文参考)	外部论文协议	— 论文值	—	—	48.48±0.34%	—	—	—

结果观察：

- 已完成的 seed 0 中，A14 的 Test BAcc 比 N14 高 1.08 个百分点，但 Test Acc 低 2.83 个百分点。
- 这表明静态 prototype 在当前单 seed 下可能改善类别平衡性，但不能据此认定总体分类已改善。
- D14 已完成 30 epochs，最高 Val BAcc 出现在 epoch 1；表中的 Test 指标都是该 epoch 对应的结果。
- 当前结果中，D14 的 Test BAcc 比 N14 低 2.49 个百分点，比 A14 低 3.57 个百分点。
- D14 使用 seed 1，A14/N14 使用 seed 0，因此这不是严格的同 seed 因果对照，不能把差距全部归因于 Dynamic correction。

21 导联结果观察：

- A21 和 N21 均为 seed 0、freeze CNN，都在 epoch 4 达到最高 Val BAcc，是当前可直接比较的一对。
- A21 的 Val BAcc 比 N21 高 0.77 个百分点 (41.84% vs. 41.07%)，但该 Val 选中 epoch 对应的 Test BAcc 反而低 1.08 个百分点 (41.00% vs. 42.08%)。
- Test Acc 基本相同：A21 58.07%，N21 58.18%，A21 低 0.11 个百分点。Kappa 也基本相同：0.4884 vs. 0.4886。
- 目前证据不支持“21 真实 + 7 prototype 优于直接使用 21 真实导联”；A21 的 Val BAcc 更高，但 N21 的 Test Acc/BAcc 更高。
- outputs/erpcore 中尚无 D21 Stage1/Stage2 输出，因此暂时不能完成 A21/N21/D21 比较。

README 中 12 导联参考的原始结论：

- 同为 freeze CNN 时，Test Acc 为 A12 61.38% > N12 60.59% > O28 57.60%，但 Test BAcc 为 O28 44.36% > A12 40.29% > N12 39.73%；因此 O > A > N 只在 BAcc 上成立。
- 同为 freeze CNN 时，A12 的 Test Acc/BAcc 都高于 N12：61.38% > 60.59%、40.29% > 39.73%。
- A12 freeze CNN 低于 N12 full finetune：Test Acc 61.38% < 64.42%，Test BAcc 40.29% < 45.42%。
- O28 full finetune 的 Test Acc/BAcc 为 64.61% / 48.91%，是表中 README 历史实验的最高值。
- D12 seed 1 低于 A12 seed 0：Test Acc 54.66% < 61.38%，Test BAcc 39.61% < 40.29%；但 seed 不同，这不是严格匹配的对照。
- CSLP 在 README 中只记录了 Test Acc 48.48±0.34%，没有用它反推缺失的 BAcc/Kappa/F1。

表内 README 的 A/N/O 来自另一个 LaBraM-unified-A0N/outputs/erpcore 历史目录，并非当前 dynamic checkout 的 outputs/erpcore。它们均为 seed 0，按 Val BAcc 选择 best checkpoint。D12 则来自当前 dynamic checkout 的 seed 1 结果。因此 12 导联历史行与 14/21 导联新实验只能并列参考；评估增加导联的效果时，应优先比较当前 checkout 中同 seed、同训练协议的实验。

证据日志：

- N14:outputs/erpcore/erp_core_N_freeze_cnn/run_logs/erp_core_N_freeze_cnn_seed0_20260914_110504.log
- A14:outputs/erpcore/erp_core_A_freeze_cnn/run_logs/erp_core_A_freeze_cnn_seed0_20260914_110732.log
- D14:outputs/erpcore/erp_core_D_stage2_14/run_logs/erp_core_D_stage2_14_seed1_20260914_115803.log
- N21:outputs/erpcore/erp_core_N_freeze_cnn_21/run_logs/erp_core_N_freeze_cnn_21_seed0_20260915_001651.1
- A21:outputs/erpcore/erp_core_A_freeze_cnn_21/run_logs/erp_core_A_freeze_cnn_21_seed0_20260915_001730.1

5.2 D Stage1 的 12 导联与 14 导联重建结果

Stage1 按最低 Val 加权总 loss 选择：

Stage1	Seed	缺失导联数	Best epoch	Val total loss	Val missing MSE	Test total loss	Test missing MSE
D12 Stage1	0	16	19	0.222727	0.006383	0.199029	0.005573
D14 Stage1	0	14	18	0.183444	0.005078	0.164856	0.004440

D14 的 missing MSE 更低是合理的，因为它比 D12 多观测 C5/C6，需要预测的导联从 16 个减少到 14 个。但这只证明 CNN token 重建误差降低，**不能直接推出分类 BAcc 一定提高**。

证据日志：

- D12 Stage1:outputs/erpcore/erp_core_D_stage1/run_logs/erpcore_D_stage1_seed0_20260914_104038.log
- D14 Stage1:outputs/erpcore/erp_core_D_stage1_14/run_logs/erpcore_D_stage1_seed0_20260914_113040.log

5.3 已完成的 D12 Stage2 参考

outputs/erpcore/erp_core_D_stage2 中有一个已完成的 12 导联 seed 1 运行：

方法	Seed	Best epoch	Val BAcc	Test Acc	Test BAcc	Test Kappa	Test Weighted F1
D12	1	23	36.60%	54.66%	39.61%	0.4445	53.96%

日志: outputs/erpcore/erp_core_D_stage2/run_logs/erp_core_D_stage2_seed1_20260831_032343.log。该结果的导联数、seed 和时间均与当前 A14/N14 不同，只作为旧 D12 参考，不纳入 A14/N14/D14 公平排名。

6. 后续比较原则

14 导联的 A/N/D 要作为受控对照，至少需要统一：

- 相同 seed；
- 相同 Train/Val/Test split；
- 相同 CNN 初始 checkpoint 与冻结策略；
- 相同 classifier 初始化、训练 epoch 和优化参数；
- Stage2 统一用最高 Val BAcc 选 checkpoint，Test 只报告该 epoch 的结果；
- D14 Stage2 必须明确加载 outputs/erpcore/erp_core_D_stage1_14/checkpoint-best.pth。

在当前证据下，可以说“14 导联降低了 D Stage1 的缺失 token MSE”；但当前 D14 seed 1 的 Test BAcc 没有超过 A14/N14 seed 0。在补齐同 seed 对照之前，不应将这个差距解释为 Dynamic correction 的确定因果效果。

7. 当前能够得到的下游结论

本节只依据 Val BAcc 选择 checkpoint 后对应的 Test 指标讨论下游效果。重建 MSE、roughness 和 t-SNE 只用于解释现象，不作为分类性能提高的证据。

7.1 增加 C5/C6 到 14 个真实导联，没有观察到下游改善

先比较不做补全的 N：

方法	Test Acc	Test BAcc	相对 N12
N12-freeze CNN (历史参考)	60.59%	39.73%	—
N14-freeze CNN	59.18%	39.06%	Acc-1.41, BAcc -0.67

把重建误差较高的 C5/C6 改成真实观测导联后，Test Acc 和 Test BAcc 均未提高。因此当前结果不支持“难补的导联改成真实观测后，分类就会改善”。更准确的解释是：**难重建的导联不一定就是对 ERP 分类最重要的导联**。

这里的 N12 来自历史 checkout，N14 来自当前 dynamic checkout。二者可以显示趋势，但不是当前 checkout 内的严格单变量对照。

7.2 增加到 21 个真实导联后，BAcc 有上升趋势，但 Acc 下降

方法	Test Acc	Test BAcc	相对 N12
N12-freeze CNN (历史参考)	60.59%	39.73%	—
N14-freeze CNN	59.18%	39.06%	Acc-1.41, BAcc -0.67
N21-freeze CNN	58.18%	42.08%	Acc-2.41, BAcc +2.35

N21 的 Test BAcc 比历史 N12 高 2.35 个百分点，但 Test Acc 低 2.41 个百分点。这个结果更像是：**增加到 21 个真实导联可能改善少数类识别或类别平衡性，但没有提高整体正确率**。由于 N12 是历史结果，目前应称为趋势，不能称为严格因果结论。

7.3 Static prototype 的下游收益不稳定

14/21 导联中，A 和 N 均为相同 seed 0、相同 freeze-CNN 协议，可以在各自导联数内部直接比较：

导联设置	N Test Acc	A Test Acc	N Test BAcc	A Test BAcc	A 相对 N
14 导联	59.18%	56.35%	39.06%	40.14%	Acc-2.83, BAcc +1.08
21 导联	58.18%	58.07%	42.08%	41.00%	Acc-0.11, BAcc -1.08

14 导联下，加入静态 prototype 后 BAcc 小幅提高，但 Acc 明显下降；21 导联下，A 的 Acc/BAcc 都没有超过 N。因此当前证据不支持“A 稳定优于 N”。随着真实导联增加、缺失导联减少，static prototype 的边际价值可能下降；在 21 导联设置中，没有观察到为了凑满 28 导联而加入 7 个 prototype 的分类收益。

7.4 当前 Dynamic correction 尚未显示下游优势

14 导联方法	Seed	Test Acc	Test BAcc
N14	0	59.18%	39.06%
A14	0	56.35%	40.14%
D14	1	58.06%	36.57%

D14 的 Test BAcc 比 N14 低 2.49 个百分点，比 A14 低 3.57 个百分点。因此当前可以说：**尚未观察到 Dynamic correction 的下游优势**。但是 D14 使用 seed 1，A14/N14 使用 seed 0，这不是严格的同 seed 对照，不能进一步断言“Dynamic correction 确定有害”。D21 尚未运行，也还不能判断真实导联增至 21 个后 Dynamic 是否会改变这一趋势。

7.5 重建变好没有转化为分类提升

D14 相比 D12 多观测 C5/C6 后，Test missing MSE 从 0.005573 降至 0.004440，相对降低约：

$$(0.005573 - 0.004440) / 0.005573 = 20.3\%$$

但 D14 的 Test BAcc 为 36.57%，没有随重建误差下降而提高。由于 D12/D14 下游结果的 seed 不同，这里不能精确归因导联数量；不过结合 A/N/D 总表，当前至少没有证据表明较低的 missing-token MSE 会自动带来更高的分类 BAcc。可以得到的稳健结论是：**缺失 CNN token 重建质量与下游判别质量不是同一个目标**。

7.6 Full finetune 的收益大于当前补全方法之间的差异

输入	Freeze CNN Test BAcc	Full finetune Test BAcc	提升
N12	39.73%	45.42%	+5.69
O28	44.36%	48.91%	+4.55

这两个提升幅度都大于当前 A/N 之间约 1 个百分点的差异，也大于历史 N12 到 N21 的 +2.35 BAcc 趋势。因此表中最强的下游信号不是某一种补全方式，而是：**ERP-Core 分类性能对特征提取器是否参与任务适配非常敏感，冻结 CNN 可能比缺失若干导联更限制下游性能**。

这里的 full-finetune 数值来自 README 历史实验，只能用于比较同一历史表内的 freeze/full 差异，不能与当前 checkout 的新实验做完全严格的归因。

7.7 可直接用于汇报的总判断

当前结果表明，导联补全质量与下游分类收益并不一致。增加 C5/C6 虽然降低了 Dynamic 缺失 token MSE，但 14 导联分类没有改善；增加到 21 个真实导联后，N 的 BAcc 出现上升趋势，但 Accuracy 下降。Static prototype 在 14 导联下只有小幅 BAcc 收益，在 21 导联下收益消失；当前 Dynamic correction 也尚未超过 A/N。相比这些差异，full finetune 带来的 BAcc 提升更明显。这说明当前瓶颈更可能涉及补全表示与分类目标不一致，以及冻结 backbone 限制了任务适配，而不只是观测导联数量不足。

按证据强度区分：

- **当前同 seed 直接支持：** A21 不优于 N21；A14 只提高 BAcc、同时降低 Acc；static prototype 的收益不稳定。
- **结合重建与下游结果支持：** missing-token MSE 下降不保证分类指标提高。
- **历史表内直接支持：** full finetune 对 BAcc 的提升明显大于 freeze 设置中 A/N 的差异。
- **仅为趋势：** 21 个真实导联可能改善 BAcc，因为 N12 与 N21 来自不同 checkout。
- **尚不能确定：** Dynamic 是否确定弱于 A/N；需要补齐相同 seed 的 D14，D21 也尚未运行。